

1. Giriş

Bu doküman DTR aşamasına gelmiş tasarımların asgari fonksiyonalitesinin test edilebilmesi için hazırladığımız ortamın yarışmacılar tarafından kendi ortamlarında çalıştırılabilmesi için hazırlanmıştır. İlerleyen kısımlarda tasarımdan beklenen asgari fonksiyonalite, test ortamının içeriği, nasıl hazırlanacağı ve çalıştırılacağı açıklanacaktır.

2. Asgari Tasarım Gereksinimleri

Testi çalıştırabilmek için tasarımınızda aşağıdaki modüllerin çalışır duruma getirilmiş olması beklenmektedir.

- CV32E40P Çekirdeği
- UART0 (YZ Hızlandırıcı UART'ına ihtiyaç yok)
- Boot ROM (QSPI'a ihtiyaç yok, varsa kullanabilirsiniz ancak testbench'i sadece boot ROM kullanarak boot edilecek şekilde kurguladığımız için QSPI kısmını bypass etmeniz işinizi kolaylaştıracaktır.)
- DATA SRAM (Koşulan test kodunun çalışabilmesi için gerekli)

3. Test Ortamının İçeriği

Yarışmacılara sağlanacak test ortamı aşağıdaki gibidir. Klasörü reponuzun içine kopyalayın ve `user_files` klasörü altındaki dosyaları 4. bölümde açıklandığı gibi güncelleyin.

```
teknotest
├── scripts
│   └── create_vivado_proj.tcl // Simülasyon ortamı için Vivado projesini
│                               // oluşturur
├── sw
│   ├── scripts
│   │   ├── build.py // C kodunu derler
│   │   └── elf_to_mem.py // Çıkan .elf dosyasını Vivado simülasyon
│   │                       // uyumlu .mem dosyasına dönüştürür
│   └── src
│       ├── crt0.S // Çekirdek entry kodu
│       └── helloworld.c // Test uygulaması
├── tb
│   └── teknotest_tb.sv // Testbench
└── user_files // Sadece buradaki dosyaları değiştirin. Açıklama aşağıda
    ├── bootrom.ld
    ├── compile_user_design.tcl
    ├── rv_toolchain.conf
    ├── teknotest_tb_user_code.sv
    ├── teknotest_wrapper.sv
    └── user_defines.h
```

ÖNEMLİ NOT: Sadece `user_files` klasörü altındaki dosyaları güncelleyin. Diğer klasörlerde bir değişiklik yapmanız hakem değerlendirmesi esnasında test ortamınızın çalışmamasına sebep olabilir. Gerçekten diğer dosyalarda bir değişikliğe ihtiyacınız olduğunu düşünüyorsanız DDK'ya danışabilirsiniz.

4. Test Ortamının Hazırlanması

Testleri doğru şekilde kořabilmek için `user_files` klasörü altındaki dosyaları kendi tasarımınıza ve ortamınıza göre düzenlemeniz gerekmektedir.

4.1. Yazılım Ortamının Hazırlanması

Yazılım ortamını ayağı kaldırabilmeniz için aşağıdaki 3 dosyanın güncellenmesi gerekir:

- `bootrom.ld`
- `rv_toolchain.conf`
- `user_defines.h`

Öncelikle `bootrom.ld` linker dosyasında aşağıda gösterilen alanda `#BOOTROM_BASE_ADDR#` ve `#DATA_RAM_BASE_ADDR#` alanlarını sırasıyla Boot ROM (veya instruction SRAM) ve data SRAM adreslerinizle değiştirin.

```
11  MEMORY
12  {
13      BOOTROM (rx) : ORIGIN = #BOOTROM_BASE_ADDR#, LENGTH = 1K
14      RAM      (rwx) : ORIGIN = #DATA_RAM_BASE_ADDR#, LENGTH = 8K
15  }
```

Örnek olarak:

```
11  MEMORY
12  {
13      BOOTROM (rx) : ORIGIN = 0x00000000, LENGTH = 1K
14      RAM      (rwx) : ORIGIN = 0x80000000, LENGTH = 8K
15  }
```

NOT: Linker dosyasında bu alan dışında bir alanda değişiklik yapmayınız.

Ortamınızdaki toolchaini kullanabilmek için `rv_toolchain.conf` dosyasını güncelleyin. `build.py` scripti bu dosyayı okur ve verilen dosya yolundaki verilen prefixe sahip toolchaini kullanır.

Aşağıda verilen örnekleri işletim sisteminize göre uncomment edip benzer formatta kendi toolchain dosya yolunuza işaret ettirebilirsiniz.

```
1  # Define your RISC-V toolchain path here
2  # Linux example:
3  # RISC_V_GCC_PREFIX = "/opt/riscv/bin/riscv32-unknown-elf"
4  #
5  # Windows example:
6  # RISC_V_GCC_PREFIX = r"C:\RISC-V\bin\riscv32-unknown-elf"
```

NOT: Diğer dosyaların aksine bu dosya hakem değerlendirmesi esnasında tekrar güncellenecektir. Bu dosyada hakemler için geçerli bir yol vermenize gerek yoktur, sizin ortamınızda çalışması yeterlidir.

NOT: `RISC_V_GCC_PREFIX` değişkenine atadığınız yol tam bir klasör yolu değildir. Bu değişken bir ön ek mantığıyla çalışır. Örneğin `gcc` derleyicinizin tam yolu Linux için `/home/my_pc/riscv_toolchain/bin/riscv32-corev-elf-gcc` olsun. Bu durumda `RISC_V_GCC_PREFIX = "/home/my_pc/riscv_toolchain/bin/riscv32-corev-elf"` olarak toolchain

yolunu belirtmeniz gerekir. Ayrıca Windows için .exe eklerini Python scripti otomatik olarak eklemektedir. Onun için ekstra bir belirtece ihtiyacınız yoktur.

Son olarak user_defines.h dosyasında UART_BASE_ADDR sabitine kendi mimarinizde UART'ın taban adresini atamanız gerekmektedir. Bu header dosyası helloworld.c içinde include edilmekte ve UART erişimleri buna göre yapılmaktadır.

4.2. Testbench Ortamının Hazırlanması

Testbench ortamını ayağa kaldırabilmeniz için aşağıdaki 3 dosyanın güncellenmesi gerekir:

- teknotest_wrapper.sv
- teknotest_tb_user_code.sv
- compile_user_design.tcl

Öncelikle tasarımınızı testbench ortamında kullanabilmek için teknotest_wrapper.sv içinde instantiate edin. Wrapper modülünün 4 pini aşağıdaki tabloda açıklanmıştır. Tasarımınız bu 4 pinin dışında pinleri içeriyorsa wrapper modülü içerisinde 0/1'e bağlanmalı ya da, dinamik olarak sürülmeleri gerekiyorsa, teknotest_tb_user_code.sv içerisinde sürülebilirler.

Pin Adı	Pin Yönü	Açıklama
clk_i	Input	Clock sinyali girişi
resetr_i	Input	Reset sinyali girişi (Active Low)
uart_rx_i	Input	UART RX Girişi (testbench->wrapper)
uart_tx_o	Output	UART TX Çıkışı (wrapper->testbench)

teknotest_tb_user_code.sv dosyası içerisinde yazdığınız kod teknotest_tb.sv içerisinde include edilir. Bu dosya içinde Boot ROM (veya instruction SRAM) içerisine test kodunu yükleyebilir ve sistemin ayağa kalkması için sürülmesi gereken sinyaller mevcutsa gerekli kod buraya yazılabilir. teknotest_tb_user_code.sv dosyasının kullanımı için örnek aşağıda verilmiştir.

```
1 // Add your memory initialization code here.
2 // Example:
3 // initial $readmemh("helloworld.mem", dut.i_bootrom.rom);
4 //
5 // You MUST KEEP "helloworld.mem" file constant, since it will be automatically
6 // added to the Vivado project, you should only change boot ROM memory array path.
7 // Also you can add processes needed for your system to boot
```

NOT: helloworld.mem dosyası create_vivado_proj.tcl scripti içerisinde Vivado projesine eklenmiş olduğundan bu dosyanın adının sabit kalması gerekmektedir. Bunun dışında memory initialization kodlarının yazımı yarışmacıların gereksinimine bırakılmıştır.

Son olarak tasarımınızı derleyebilmek için compile_user_design.tcl dosyasını düzenlemeniz gerekmektedir. Bu dosyada tasarım dosyalarınızı projeye ekleyebilir, include dosyalarını tanımlayabilir ve kod içerisinde kullanmanız gereken define macroları varsa tanımlayabilirsiniz. compile_user_design.tcl dosyasının kullanımı için örnek aşağıda verilmiştir.

```

1 # Add your design files into the Vivado project
2 #
3 # To add design files, use
4 # add_files <PATH_TO_YOUR_DESIGN_FOLDER>
5 # Example:
6 # add_files ../user/rtl/teknotest_wrapper.sv
7 #
8 # To add include directories, use
9 # set_property include_dirs [file normalize <PATH_TO_YOUR_INCLUDE_FOLDER>] [list [get_filesources_1] [get_filesources_sim_1]]
10 # Example:
11 # set_property include_dirs [file normalize "../user/rtl/core/include/"] [list [get_filesources_1] [get_filesources_sim_1]]
12 #
13 # If you need to add verilog defines, use
14 # set_property verilog_define {<YOUR_DEFINES>} [list [get_filesources_1] [get_filesources_sim_1]]
15 # Example:
16 # set_property verilog_define {SIMULATION USE_CG_BEHAV_MODELS} [list [get_filesources_1] [get_filesources_sim_1]]
17 #
18 # Note1: Keep in mind that you will be in teknotest directory, set your file/folder paths relative to that directory
19 # Note2: You can use slashes(/) in Windows too to separate the paths

```

ÖNEMLİ NOT: Dosya pathi verirken absolute path yerine teknotest klasörüne göre relative path verilmesi gerekmektedir. Absolute path içeren scriptler hakem değerlendirmesi sırasında çalışmayacağından, testi geçememiş sayılmanıza sebep olacaktır.

5. Testin Koşturulması

Koşturulacak olan test senaryosu UART RX ve TX hatlarını test edecektir. Test akışı aşağıdaki gibi ilerlemektedir.

0. Testbench tarafından wrapper modülüne 50 MHz clock ve reset sinyalleri verilerek test edilecek modül ayağa kaldırılır. (C kodu 50 MHz clocka göre baud rate hesapladığından, tasarım içinde PLL mevcutsa o da 50 MHz clock çıkaracak şekilde konfigürasyon yapmayı unutmayın.)
1. CPU UART kontrolcüsünü ayağa kaldırır ve 'R' karakterini TX hattından göndermeye başlar.
2. Testbench tarafında 'R' karakterinin gelmesi beklenir ve karakter geldikten sonra 'A' karakteri gönderilir.
3. CPU 'A' karakterini bekler, karakter geldikten sonra "Hello World!" mesajını göndermeye başlar.
4. Testbench "Hello World!" mesajını bekler. Burada 3 ayrı senaryo gerçekleşebilir.
 - a. "Hello World!" mesajı beklendiği gibi gelir ve test başarıyla sonuçlanır.
 - b. "Hello World!" mesajı ile aynı uzunlukta farklı bir mesaj alınırsa beklenen mesaj ve gelen mesajın içeriği ekranda bastırılır ve test başarısız olarak sonuçlanır.
 - c. Testbenchte belirlenen 10 ms süre içerisinde "Hello World!" mesajı veya onunla aynı uzunlukta bir mesaj gelmezse test timeout olur ve başarısız olarak sonuçlanır.

Testi koşturabilmek için öncelikle test yazılımının derlenmesi gerekmektedir. Terminal açıp `sw` klasörü altında `scripts/build.py` scriptini çalıştırarak test yazılımını derleyebilirsiniz. Aşağıda örnek komut verilmiştir.

```
PS C:\Users\bcakar\Documents\teknotest_env\teknotest\sw> & C:/Users/bcakar/AppData/Local/Programs/Python/Python39/python.exe ./scripts/build.py
```

Her şey yolunda gittiği takdirde derlemenin başarılı olduğuna dair aşağıdaki gibi bir mesaj görünecektir.

```

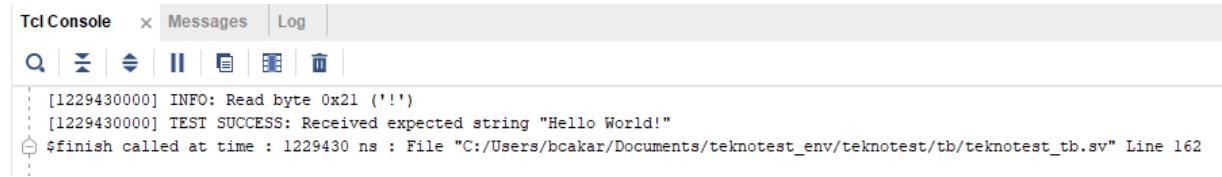
Build completed successfully.
ELF      : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.elf
MAP      : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.map
DISASM   : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.dis
READELF  : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.readelf
MEM      : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.mem

```

Test yazılımını derledikten sonra Vivadoyu açıp TCL konsolunda aşağıdaki komutları koşturun. İlk komutu çalıştırmadan önce `<teknotest_folder_path>` değişkenini kendi sisteminizde `teknotest` klasörünün yolu ile değiştirin.

```
cd <teknotest_folder_path>
source ./scripts/create_vivado_proj.tcl
```

İkinci komutta çalıştırılacak olan script otomatik olarak Vivado projesini oluşturup projeyi açacaktır. Projenin açılmasının ardından simülasyonu sonuna kadar koşturun. Başarılı bir simülasyon aşağıdaki çıktıyı vermelidir.



The screenshot shows the 'Tcl Console' window with tabs for 'Messages' and 'Log'. The console output is as follows:

```
[1229430000] INFO: Read byte 0x21 ('!')
[1229430000] TEST SUCCESS: Received expected string "Hello World!"
$finish called at time : 1229430 ns : File "C:/Users/bcakar/Documents/teknotest_env/teknotest/tb/teknotest_tb.sv" Line 162
```